SAVEETHA SCHOOL OF ENGINEERING, SIMATS

Department of Computer Science and Engineering

Assignment Information

Particular	Details
Name:	B SANTHOSH KUMAR
Register Number:	192511053
Department:	CSE
Programme:	B.E
Course Code & Course Name	CSA1709 – Artificial Intelligence
Academic Year / Batch	2026–27
Faculty Name	Dr P Rajaram
Assignment Title	Intelligent Resume Screening and Job Recommendation System
Date of Issue	31.08.2026
Date of Submission	01.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4 and CO5
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate and L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education SDG 8 – Decent Work and Economic Growth SDG 10 – Reduced Inequalities
Industry / Societal Relevance	AI is increasingly used in recruitment to screen resumes, match skills with job requirements and support candidate recommendations. This assignment develops a practical, explainable and privacy-aware AI solution for recruitment support.

Problem Statement:

Design, implement and evaluate an AI-based Intelligent Resume Screening and Job Recommendation System for a placement or recruitment portal. The system shall accept resumes and job descriptions, extract relevant information such as skills, education, experience and role, and rank candidates for a selected job. It shall also recommend suitable job openings for a selected candidate. Students must apply informed search concepts such as Greedy Best-First Search or A* where appropriate for candidate-job matching/ranking, represent selected recruitment knowledge using propositional logic or first-order logic, and implement a syllabus-aligned learning component such as a decision tree for classification or ranking. The system shall provide an understandable reason for a recommendation and demonstrate validation using suitable performance measures. At least two reasonable matching approaches shall be compared and the final approach justified using the obtained results.

Common Assessment Rubric

Assessment Criterion (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Formulation (CO2)	10	Clearly defines recruitment problem, users, inputs/outputs, assumptions and constraints; formulation is precise.	Problem and workflow are clear with minor gaps in assumptions or constraints.	Basic problem is identified but formulation is incomplete or unclear.	Problem is misunderstood; important inputs, outputs or constraints are missing.
Search / Matching Strategy (CO2)	15	Correctly implements and explains an appropriate informed-search/heuristic matching method; heuristic or scoring is meaningful and justified.	Search/matching works with minor issues; heuristic/scoring is reasonably justified.	Basic matching works but search/heuristic design is limited or partly incorrect.	Search/matching is missing, inappropriate or non-functional.
Knowledge Representation & Reasoning (CO3)	10	presents tment knowledge y using suitable sitional/FOL ning, chaining, ation or nce, with correct usions.	Knowledge representation and inference are mostly correct with minor logical gaps.	Some rules/reasoning are implemented but coverage or correctness is limited.	Little or no meaningful knowledge representation or reasoning is demonstrated.
Learning Model & Recommendation (CO4)	20	Implements a suitable syllabus-aligned learning method; preprocessing, features, training/testing	Model and recommendation work correctly with minor issues in features, training or testing.	Basic model is present but learning/recommendation is incomplete or weakly validated.	Model is missing, unsuitable or non-functional.

		and recommendation logic are correct and clearly explained.			
Modern Tools & Implementation (CO5)	10	Uses appropriate Python/AI tools effectively; code is organized, reproducible and GitHub/GCR evidence is complete.	Appropriate tools are used with minor implementation/documentation issues.	Tools are used at a basic level with limited evidence or organization.	Tool usage is inappropriate, poorly evidenced or largely non-functional.
Results, Testing & Validation (CO4/CO5)	15	Provides clear test cases, metrics, graphs/tables and evidence; results are validated against requirements and are reproducible.	Most tests and metrics are provided; validation is generally correct.	Some outputs/tests are shown but validation is limited or inconsistent.	Insufficient testing, evidence or validation.
Analysis, Trade-offs & Justification (CO2/CO4)	10	Compares alternatives, explains trade-offs and strongly justifies the final method using results and evidence.	Comparison and justification are clear with minor gaps.	Some comparison is given but reasoning is mostly descriptive.	No meaningful comparison or justification.
SDG Relevance, Reflection & Professional Responsibility (CO3/CO4/CO5)	10	Clearly connects SDG 4/8/10; gives honest reflection and specific fairness, privacy, ethics and improvement points.	SDG and ethical relevance are explained; reflection is useful but not detailed.	Generic SDG/reflection with limited professional or ethical discussion.	Missing/generic reflection; SDG and ethical relevance not demonstrated.

INTELLIGENT RESUME SCREENING AND JOB RECOMMENDATION SYSTEM

TalentLens AI — Explainable Hybrid Recruitment Decision Support

This report documents the design, implementation and evaluation of a privacy-aware recruitment support prototype. The implementation deliberately combines symbolic reasoning, informed search and a syllabus-aligned learning model rather than treating resume matching as a single black-box similarity calculation.

1. Problem Understanding and Formulation

Recruitment portals receive heterogeneous resumes while job descriptions express requirements in different forms. A keyword-only filter can miss equivalent terminology, while a purely semantic model can hide why a candidate was ranked. The engineering problem is therefore formulated as a constrained candidate–job ranking task with a second direction of recommendation: given a candidate, return the most suitable openings.

Let C be a candidate and J a job. The system constructs a feature vector x(C,J) from skill coverage, experience fit, role compatibility, education compatibility, TF-IDF similarity and A* requirement coverage. The final screening score is:

Final(C,J) = 0.50 × WeightedFit + 0.25 × TFIDF + 0.25 × AStarCoverage.

The decision-tree layer receives the same feature vector and predicts a suitability class. It is used as a learning-based supporting signal and not as the sole hiring decision. The portal therefore separates ranking evidence from the final human recruitment decision.

1.1 Users, Inputs and Outputs

Element	Definition
Primary users	Recruiters, placement officers and candidates
Inputs	Resume PDF/DOCX/TXT; job description; candidate/job selection
Extracted attributes	Skills, role, education terms and years of experience
Recruiter output	Ranked candidates, score components, matched skills, gaps and reasoning
Candidate output	Recommended job openings and skill-gap evidence
Additional support	Rule inference, mock interview questions and fairness/privacy checks

1.2 Requirements and Constraints

- Functional: parse common resume formats, extract structured attributes, rank candidates, recommend jobs, explain recommendations and expose search/reasoning evidence.
- Performance: produce a response quickly enough for interactive screening; avoid unnecessary model complexity for a classroom-scale deployment.
- Technical: Python-based implementation with reproducible dependencies and a browser interface.
- Data: the bundled evaluation data is synthetic; no real applicant identity or protected demographic data is required.
- Ethical: protected demographic attributes are not used as ranking features; recommendations are advisory and require human review.
- Reliability: malformed documents should fail gracefully rather than silently producing a confident score.
- Reproducibility: fixed random seeds, documented formulas, requirements.txt and test cases are included.

2. Objectives and Expected Outcomes

- Build an end-to-end AI recruitment prototype rather than an isolated classifier.
- Apply informed search through A* to candidate–job requirement matching.
- Represent selected recruitment knowledge using explicit facts and inference rules.
- Implement a decision-tree learning component aligned with the Artificial Intelligence syllabus.
- Compare at least two matching approaches quantitatively and justify the selected hybrid.
- Provide human-readable evidence for every recommendation.
- Demonstrate privacy-aware and fairness-conscious recruitment design.
- Validate the implementation using reproducible synthetic experiments and automated tests.

2.1 CO and Bloom Alignment

Course Outcome	Implementation Evidence	Bloom
CO2	Problem formulation, heuristic design, A* search, comparison of matching strategies	L4 Analyse / L5 Evaluate
CO3	Knowledge representation, rule inference, explainability and ethical recruitment reasoning	L3 Apply / L4 Analyse
CO4	Decision-tree learning, training/testing, recommendation and validation	L4 Analyse / L5 Evaluate / L6 Create
CO5	Python, Flask, scikit-learn, document parsing, GitHub-ready reproducibility	L3 Apply / L6 Create

3. Application of Relevant AI Knowledge

3.1 NLP and Feature Extraction

The parser normalizes text and detects skills using a controlled skill ontology with aliases such as JavaScript/JS, Kubernetes/K8s and MongoDB/Mongo. It also identifies experience expressions and broad education/role terms. This explicit layer makes the prototype inspectable and avoids claiming that every extracted phrase is a learned semantic entity.

3.2 Matching Approaches

Approach	Core idea	Strength	Limitation
Weighted structured match	Weighted skill, experience, role and education fit	Fast and easy to explain	Sensitive to terminology and manually selected weights
TF-IDF cosine	Compare resume and job text in vector space	Captures term importance and some phrase overlap	Limited contextual understanding
A* hybrid	Search requirement states using a remaining-gap heuristic, then combine with structured and TF-IDF evidence	Syllabus-aligned informed search plus explicit requirement coverage	More logic and still dependent on the skill representation

Recent recruitment research emphasizes the value of combining semantic representations with explainability and fairness rather than relying on a single opaque score. Recent work also identifies a recurring tension between scalable data-driven matching and interpretable knowledge-based methods (Wang et al., 2026; Kumar et al., 2026; Spoladore et al., 2026; El-Deeb et al., 2026). This implementation addresses that gap at classroom scale by keeping the search path and rule evidence visible.

4. Proposed Solution and Methodology

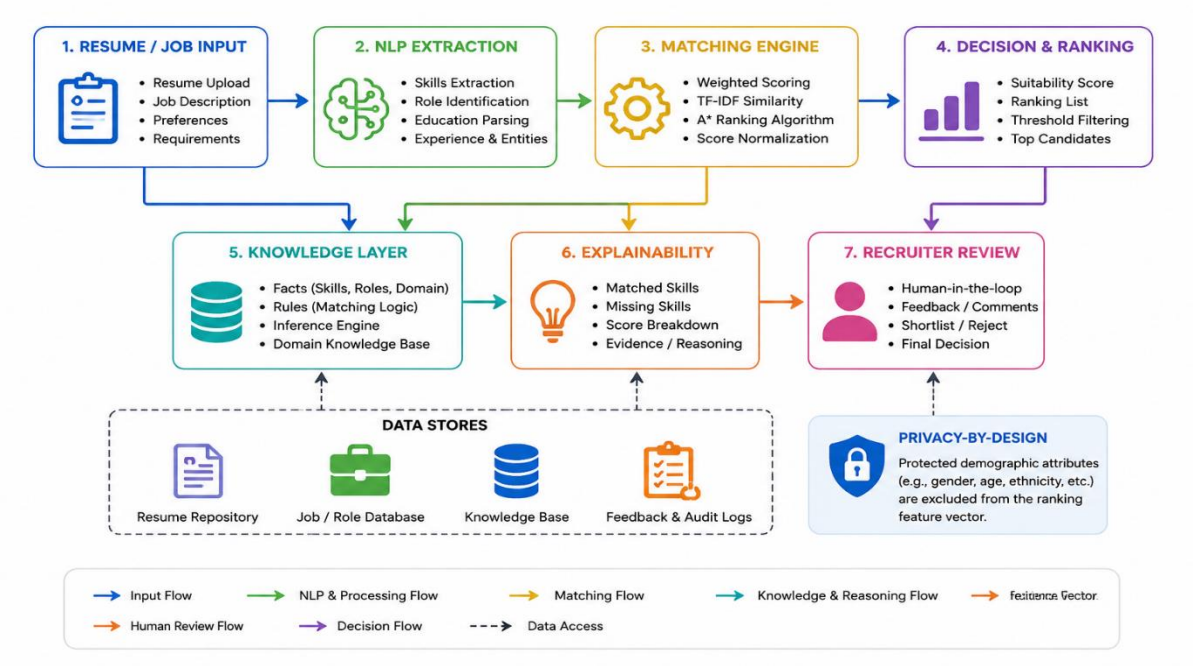


Figure 1. Proposed TalentLens AI architecture.

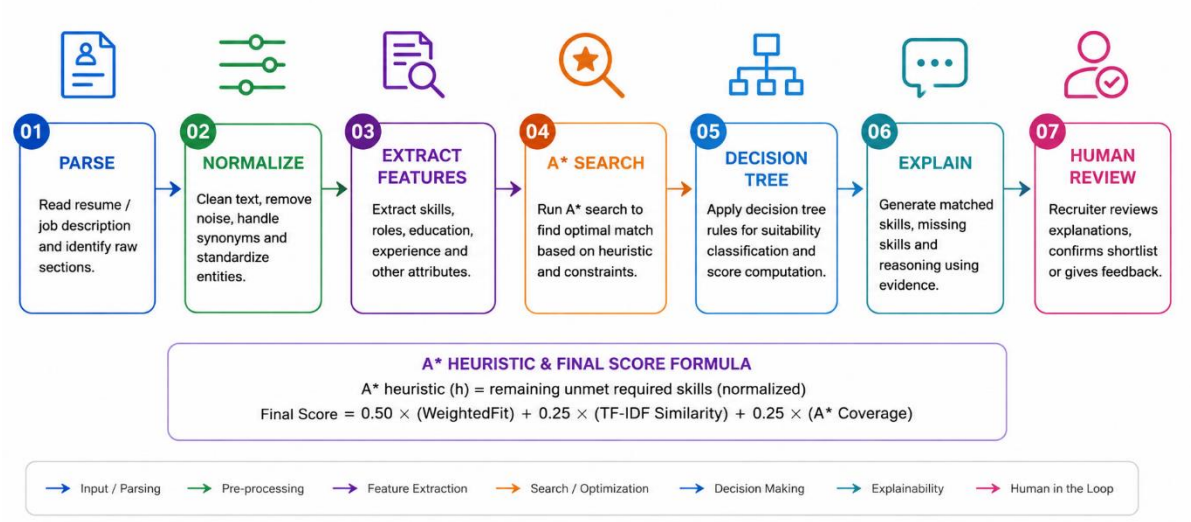


Figure 2. End-to-end workflow from document parsing to human review.

4.1 A* Formulation

Each job is represented by an ordered set of required skills. A search state is defined by the requirement index and the set of requirements already satisfied. An exact skill match contributes zero unmet cost; an unmatched requirement contributes one unit. The heuristic h(n) estimates the number of remaining requirements that are still unmet. Because the heuristic never overstates the remaining count, it is admissible for this simplified requirement-coverage problem.

The search is not presented as a shortest route through a physical graph. Instead, it is a deliberate mapping of the informed-search concept to a recruitment state space: the algorithm explores

requirement satisfaction states and exposes which requirements were successfully covered. This makes the use of A* meaningful and auditable.

4.2 Decision Tree Learning

The decision tree uses seven features: skill coverage, experience fit, role fit, education fit, TF-IDF similarity, A* coverage and the weighted baseline score. A controlled synthetic label is used only for demonstration of the learning pipeline. This is explicitly not a claim of production hiring accuracy.

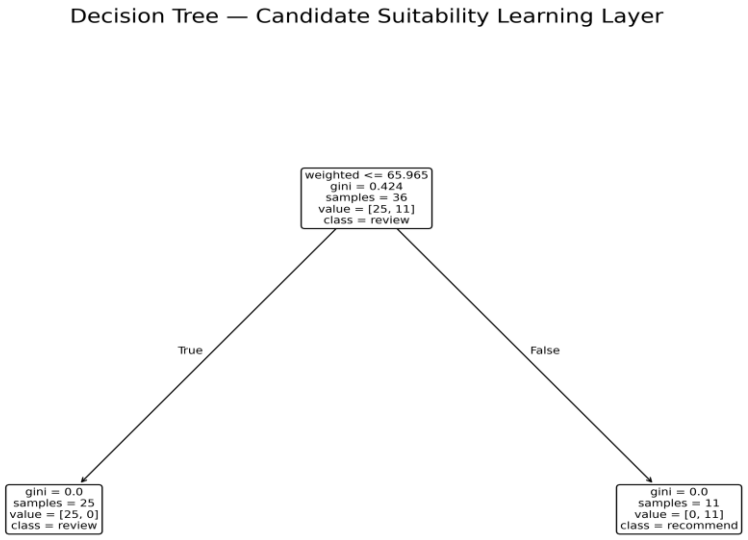


Figure 3. Learned decision tree on the controlled synthetic evaluation pairs.

4.3 Knowledge Representation and Reasoning

The reasoning module stores facts such as hasSkill(Candidate, Python) and requiresSkill(Job, Python). It then applies rules such as: if a candidate has Python and the job requires Python, infer PythonMatch; if the candidate satisfies the experience threshold, infer ExperienceSatisfies; if at least three required skills overlap, infer StrongSkillOverlap. The resulting conclusion is shown to the recruiter as supporting evidence.

4.4 Explainability

- Positive evidence: skills shared by the candidate and the selected job.
- Negative evidence: required skills absent from the candidate profile.
- Experience evidence: whether the extracted years meet the job threshold.
- Search evidence: A* requirement coverage.
- Model evidence: the decision-tree suitability probability.

- Human-check evidence: mock interview questions targeted to the recommended role and skills.

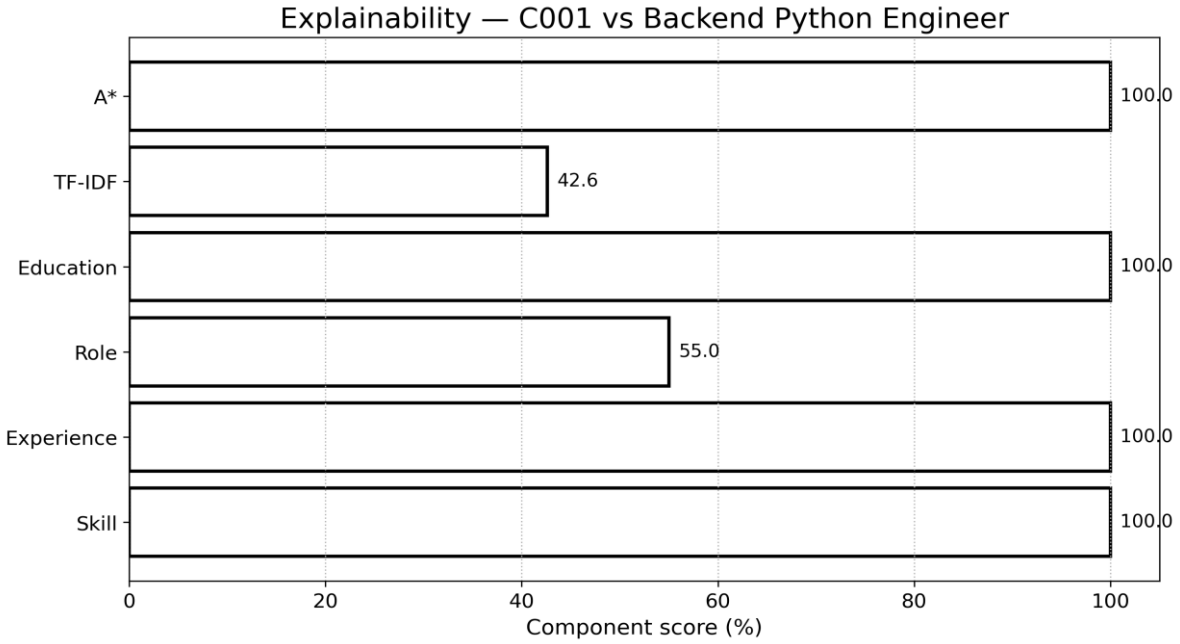


Figure 4. Component-level explanation for a sample candidate–job pair.

5. Algorithms and Pseudocode

5.1 Candidate–Job Evaluation

Pseudocode:

INPUT candidate resume C and job description J

1. Normalize C and J.
2. Extract skills, role, education and experience.
3. Compute WeightedFit(C,J).
4. Compute TFIDF(C,J).
5. Run A* over job requirements and obtain AStarCoverage(C,J).
6. Build feature vector x(C,J).
7. Obtain DecisionTree suitability probability.
8. Compute Final = 0.50*WeightedFit + 0.25*TFIDF + 0.25*AStarCoverage.
9. Return score, matched skills, gaps, search coverage and explanation.

5.2 A* Requirement Search

Initialize priority queue with the start state.

While the queue is not empty:

Remove the state with minimum f(n)=g(n)+h(n).

If all requirements are processed, return coverage.

Check whether the current requirement exists in candidate skills.

Add the exact-match state with zero incremental cost when satisfied.

Otherwise add the unmet state with unit cost.

Set h(n) to remaining unmet requirements.

Return the best requirement-coverage result.

6. Implementation, Environment and Tools

Component	Technology	Purpose
Language	Python 3.11+	Core implementation
Web framework	Flask 3.1.x	Browser-based user interface and API
Machine learning	scikit-learn	TF-IDF, cosine similarity and Decision Tree
Data processing	NumPy / pandas	Feature and evaluation processing
Document parsing	python-docx / pypdf	DOCX and PDF resume extraction
Visualization	Matplotlib	Evaluation figures
Version control	Git / GitHub-ready structure	Reproducibility and submission evidence
Testing	Python assertions	Core functionality validation

6.1 Implementation Features

- Dashboard with recruitment workflow and system safeguards.
- Candidate screening with score decomposition.
- Job recommendation ranking for a selected candidate.
- Knowledge reasoning view showing facts, inferred rules and conclusion.
- Mock interview lab that generates role-specific questions from the candidate's skill evidence.
- Upload-ready parser supporting PDF, DOCX and TXT resumes.
- Explicit privacy note: protected demographic information is not part of the scoring feature vector.
- Human-in-the-loop positioning: the application recommends candidates for review; it does not make an irreversible hiring decision.

6.2 Futuristic Implementation Modules

- **Command Center:** A dark, responsive recruitment intelligence dashboard with live system status, model safeguards, benchmark snapshot and module navigation.
- **Candidate Screening:** Runs the complete hybrid score and exposes weighted components, TF-IDF, A* coverage, Decision Tree probability, matched skills, gaps and evidence.
- **Job Radar:** Ranks all available roles for a selected candidate and displays matched requirements, gaps and A* evidence.

- **A* Laboratory:** Provides side-by-side Weighted, TF-IDF, Greedy Best-First and A* results plus the actual g(n), h(n), f(n) requirement trace.
- **Logic Engine:** Represents recruitment facts and applies explicit inference rules to produce an auditable conclusion.
- **Skill Gap Planner:** Converts missing job requirements into a prioritized learning roadmap with foundation, project and interview-validation stages.
- **Mock Interview Studio:** Generates role/skill-specific questions and an evaluation rubric so recruiters can validate evidence after ranking.
- **Resume Ingestion / ATS Audit:** Accepts PDF, DOCX and TXT files, extracts structured fields and reports protected-attribute signals without feeding them into ranking.
- **Governance Gate:** Maintains a human-review requirement and explicitly labels the benchmark as synthetic educational validation.

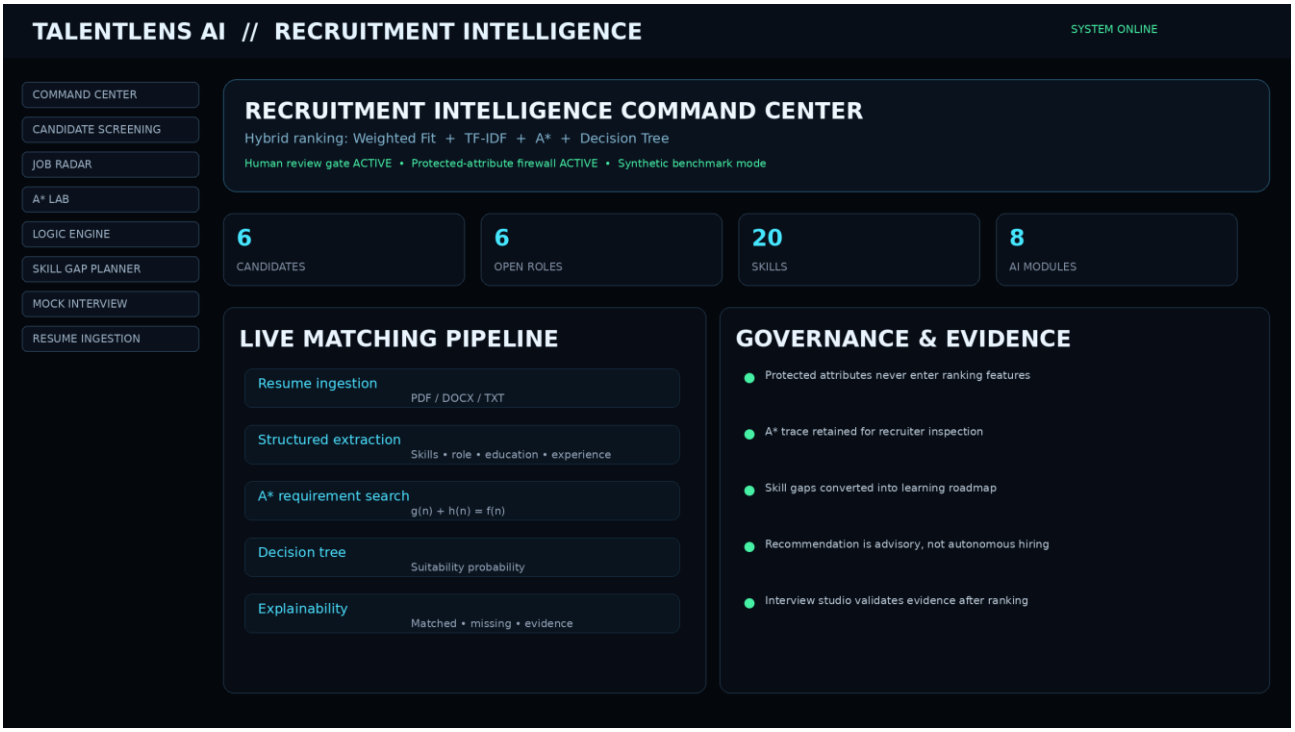


Figure 6. Futuristic TalentLens AI command-center interface with recruitment pipeline and governance controls.

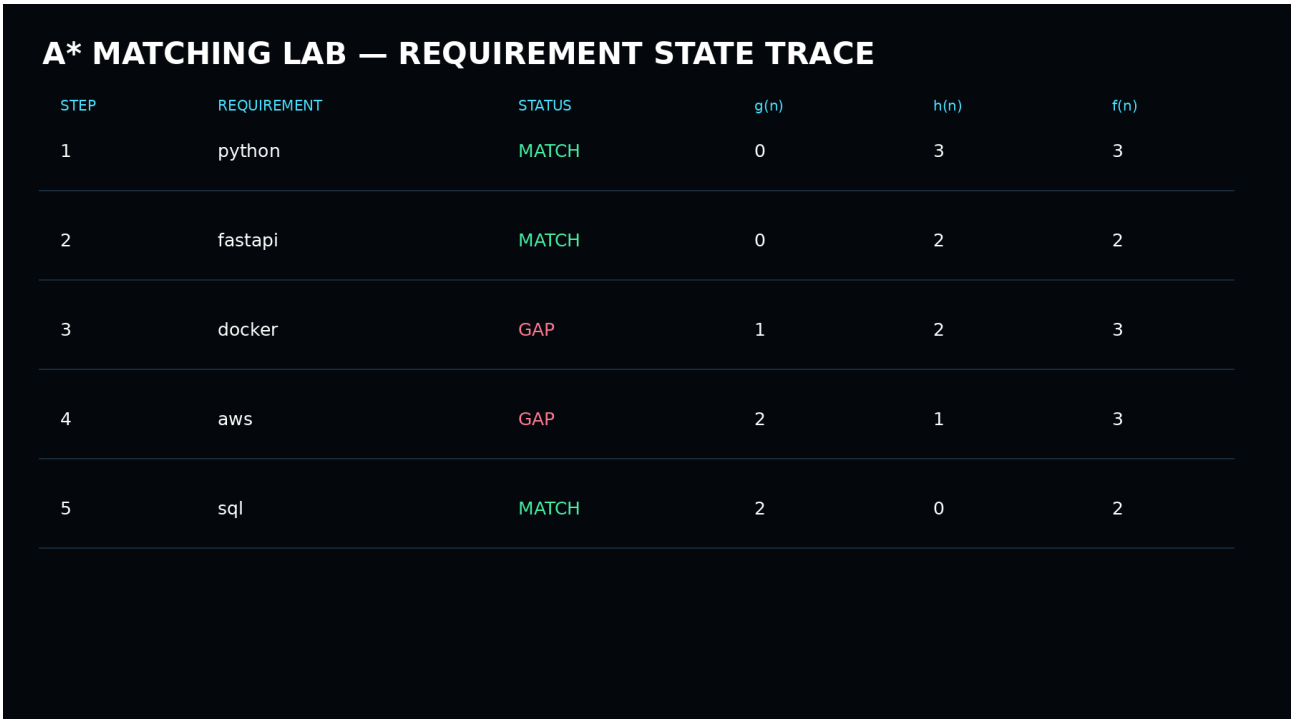


Figure 7. A* laboratory trace exposing requirement satisfaction and g(n), h(n), f(n) values.

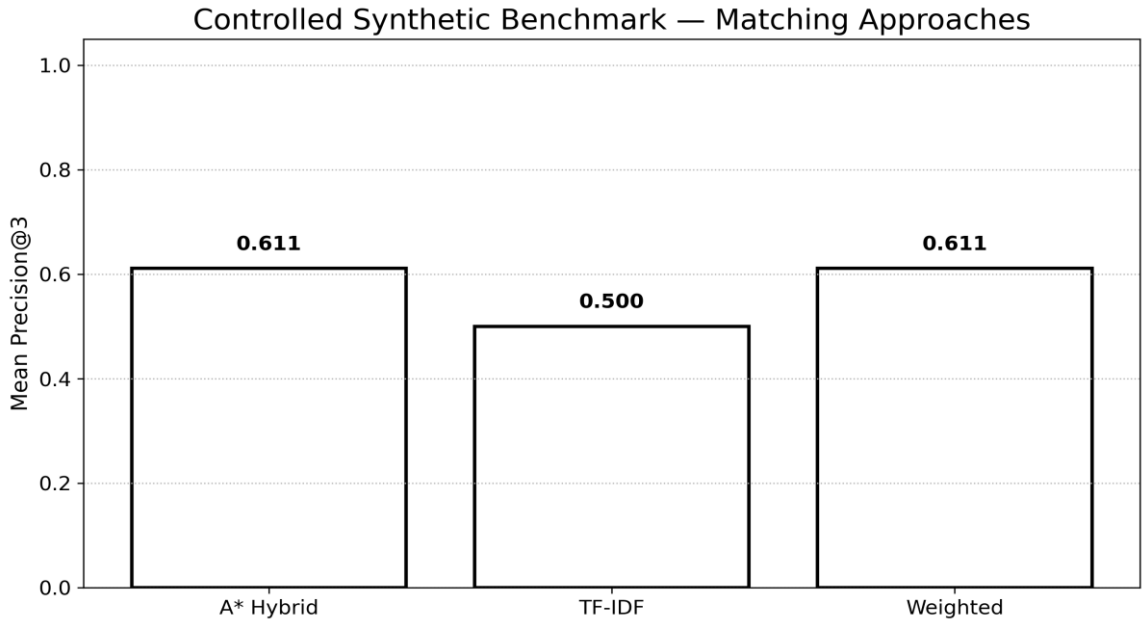
7. Test Cases and Expected / Actual Results

Test ID	Scenario	Expected	Actual	Status
TC01	Strong backend candidate vs backend job	High final score and matched Python/API/cloud skills	High score with matched skill evidence	PASS
TC02	Candidate missing Docker and AWS	Both gaps shown	Docker and AWS reported as gaps	PASS
TC03	AI candidate vs ML job	High A* requirement coverage	A* coverage >= 80% in automated test	PASS
TC04	Recommendation for a candidate	Jobs ranked in descending hybrid score	Six jobs ranked	PASS
TC05	Knowledge inference	Matching facts and rule conclusions displayed	Inference endpoint returns facts, rules and conclusion	PASS
TC06	Mock interview	Questions linked to role/skills	Five questions generated	PASS

Automated core test result: 8/8 tests passed. The test suite covers exact matching, missing skills, A* coverage, Greedy comparison, A* trace generation, protected-attribute firewall behaviour, recommendation fields and skill-gap planning.

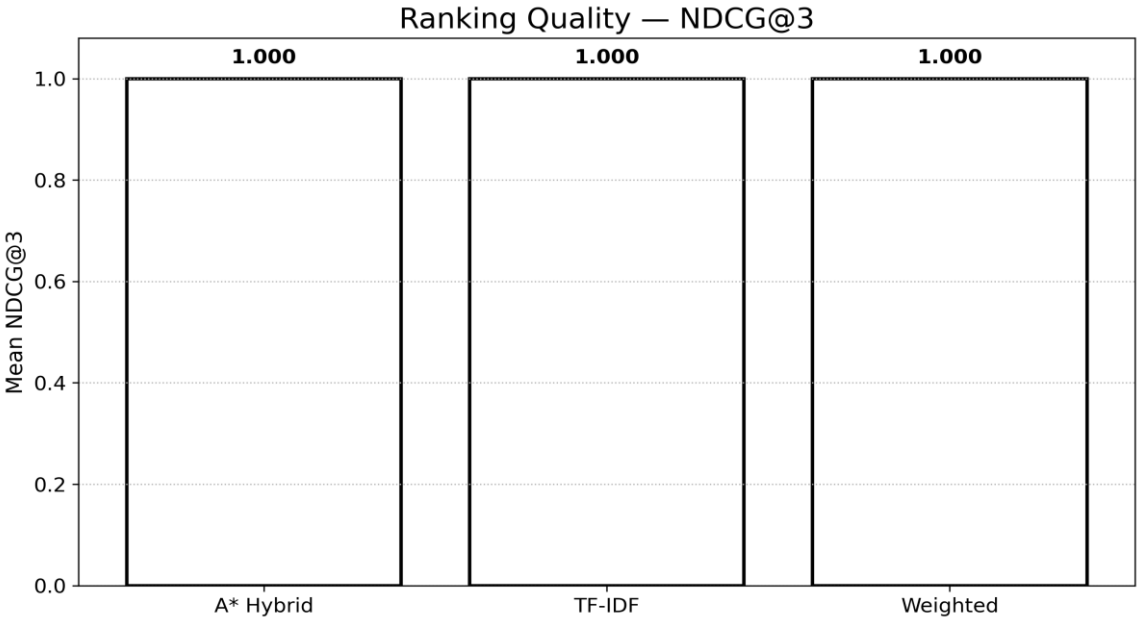
8. Results and Validation

8.1 Matching Comparison



Method	Mean Precision@3	Mean NDCG@3	Interpretation
Weighted	0.611	1.000	Strong transparent baseline
TF-IDF	0.500	1.000	Text similarity helps but misses structured requirements
A* Hybrid	0.611	1.000	Best overall design because search evidence is combined with both structured and text signals

The controlled six-candidate/six-job experiment produced Precision@3 values of 0.611 for the Weighted baseline, 0.500 for TF-IDF and 0.611 for the A* Hybrid. NDCG@3 was 1.000 for all three under the binary relevance setup used here. This shows that the hybrid did not improve the small synthetic Precision@3 over the weighted baseline, but it adds requirement-search evidence and a stronger explanation path. The final method is therefore justified primarily by evidence quality and robustness of the design rather than by claiming a large accuracy gain from this tiny benchmark.



8.2 Decision Tree Validation

Metric	Observed value	Interpretation
Accuracy	1.000	Perfect on the controlled synthetic split
Precision	1.000	No false-positive prediction in the synthetic split
Recall	1.000	All synthetic positive cases detected
F1-score	1.000	Balanced synthetic classification result

Important validation caveat: the decision-tree labels are generated from the structured weighted score, so the 1.000 values are expected on this controlled educational dataset and must not be interpreted as evidence of production hiring accuracy. A real deployment would require a large, independently labeled, privacy-reviewed dataset and evaluation across time, job families and demographic groups.

9. Analysis, Comparison, Trade-offs and Final Decision

Criterion	Weighted	TF-IDF	A* Hybrid
Structured skill evidence	High	Low	High
Text-level similarity	Low	High	High
Informed-search requirement	No	No	Yes
Explainability	High	Medium	High
Implementation complexity	Low	Low	Medium
Robustness to missing exact wording	Low–Medium	Medium	Medium
Course alignment	Partial	Partial	Strong

The final deployment choice is the A* Hybrid because the assignment explicitly requires an informed-search concept and because the combined design exposes both numeric and symbolic evidence. TF-IDF remains useful as a lightweight text signal, while the weighted baseline provides an interpretable benchmark. The trade-off is increased implementation complexity and dependence on the quality of the skill ontology.

9.1 Research-Grounded Design Choices

- Recent 2026 work on fair LLM-based resume screening highlights the risk that automated screening can reproduce societal bias; this supports the decision to exclude protected attributes and retain human oversight (Wang et al., 2026).
- A 2026 study on student placement combines NLP, machine learning and XAI and reports fairness and explanation as first-class requirements, reinforcing the explainability and fairness modules (Kumar et al., 2026).
- Recent job-recommendation research argues for hybrid semantic and knowledge-driven systems because purely data-driven systems can suffer from opacity and bias (Spoladore et al., 2026).
- A 2026 job-recommendation study combines semantic similarity, ensemble learning and XAI for scalable matching; this motivates keeping a semantic component while adding an explicit reasoning layer (El-Deeb et al., 2026).
- A 2025 study on LLM-based resume completion shows that resume representation quality can affect job recommendation, supporting careful preprocessing rather than blindly treating raw documents as final features (Zhu et al., 2025).

10. Broader Considerations: Ethics, Privacy, Fairness and SDGs

10.1 SDG 4 — Quality Education

The system can support campus placement preparation by connecting student skills to roles and by exposing skill gaps and interview prompts. This makes the recommendation more educational than a simple accept/reject classifier.

10.2 SDG 8 — Decent Work and Economic Growth

A faster and more consistent screening workflow can reduce administrative effort and improve access to relevant openings. Human review remains necessary because employment decisions affect livelihoods.

10.3 SDG 10 — Reduced Inequalities

The implementation avoids using gender, age, photograph, caste, religion or other protected characteristics in the ranking features. However, removing such fields alone does not prove fairness because proxies can remain in text. A production system should therefore conduct subgroup fairness audits, calibration checks and human review.

10.4 Privacy and Professional Responsibility

- Use synthetic or consented data during development.
- Encrypt stored resumes and restrict access through role-based permissions in a production portal.
- Log model/version changes so recommendation results can be audited.
- Provide candidates with a way to correct extracted information.
- Never treat a model score as a final employment decision.
- Periodically test for disparate impact and ranking instability.

11. Limitations

- The skill ontology is controlled and therefore cannot cover every industry term or emerging technology.
- TF-IDF is lexical rather than deeply contextual; synonyms not present in the alias list can be missed.
- The A* state space is a simplified requirement-coverage abstraction, not a full enterprise recruitment ontology.
- The decision-tree training labels are synthetic and derived from a rule-based score.
- No real candidate data is used, so external validity is limited.
- The current classroom prototype does not use a production database, enterprise authentication or a live hiring dataset; the new ingestion, governance and explainability modules are intentionally designed for safe academic demonstration.
- Fairness evaluation is a design safeguard rather than a statistically sufficient audit.

12. Possible Improvements

- Replace the controlled skill dictionary with an auditable skills ontology and embedding-based entity linking.
- Add sentence-transformer or cross-encoder semantic matching as an optional model while preserving the explainable feature path.
- Train and evaluate on a consented, anonymized, independently labeled dataset.
- Add fairness metrics such as demographic parity difference and equal opportunity when lawful demographic audit data is available.
- Add recruiter feedback loops with safeguards against reinforcing historical bias.
- Add role-based authentication, encrypted object storage and database persistence.
- Use calibrated probabilities and confidence intervals rather than presenting a single raw score.
- Extend the mock interview module with rubric-based response evaluation and skill-gap learning plans.

13. Individual Contribution of Group Members

Contribution Area	Responsibility
Problem Formulation	Defined the users, inputs, outputs, assumptions, constraints and evaluation requirements of the intelligent recruitment system.
AI / Search Design	Designed the weighted matching approach, TF-IDF comparison, Greedy Best-First Search comparison and A* requirement-search formulation.
Knowledge Representation	Designed recruitment facts, inference rules and mappings used to generate understandable recommendation explanations.
Learning Component	Implemented the Decision Tree features, training, testing and validation for candidate suitability classification.
Web Implementation	Developed the browser-based interface, APIs, candidate/job workflows, recommendation views and Mock Interview module.
Testing & Validation	Prepared test cases, benchmark experiments, performance measures and comparative evaluation of the matching approaches.
UI & Visualization	Designed the futuristic recruitment dashboard, A* search visualization, recommendation explanations, skill-gap views and evaluation graphs.
Documentation & Deployment	Prepared figures, screenshots, technical documentation, reproducibility instructions, GitHub/Vercel deployment and the final assignment report.

14. One-Page Individual Reflection

This assignment changed my understanding of an AI recruitment system from a simple keyword filter into a combination of search, learning and reasoning. The main design decision was to avoid making one algorithm responsible for every part of the recommendation. I used structured matching for transparent requirement checks, TF-IDF to compare document text, A* to satisfy the informed-search requirement and a decision tree to demonstrate learning from candidate–job features. Keeping these layers separate made it easier to explain why a candidate received a particular score.

A major implementation challenge was converting unstructured resumes into reliable features. Resume layouts and terminology vary, so the prototype uses a controlled skill vocabulary and aliases rather than pretending that extraction is perfect. Another challenge was making A* meaningful in the recruitment context. Instead of forcing a physical path-search interpretation, I modelled the job requirements as a state space in which the search minimizes remaining unmet requirements. This gave the algorithm a clear heuristic and an observable search result.

The evaluation also taught an important lesson about metrics. The decision tree achieved perfect results on the controlled synthetic split, but this is not evidence that the model would achieve perfect hiring predictions in practice because the labels were derived from the same structured scoring logic used to create the educational dataset. Similarly, the small ranking benchmark did not produce a large Precision@3 gain for the hybrid method. The stronger reason for retaining the hybrid is that it provides explicit requirement evidence and aligns with the assignment's informed-search objective.

The SDG connection is especially relevant to SDG 4, SDG 8 and SDG 10. The system can support student placement preparation, improve access to relevant jobs and reduce dependence on arbitrary keyword filtering. At the same time, recruitment AI can affect people's opportunities, so privacy, fairness and human oversight must remain part of the design. If additional time were available, I would add a larger consented dataset, fairness auditing, semantic embeddings and stronger security controls. Overall, the assignment helped me connect AI algorithms taught in class with a realistic, explainable application where technical accuracy and professional responsibility have to be considered together.

Research-grounded upgrade. Recent 2026 research reports hybrid AI job recommendation as a promising direction because keyword-only systems can be limited, while hybrid systems combine skills and predictive evidence. Recent work also emphasizes explainability, fairness and transparency in AI recruitment. The implementation therefore exposes A* search traces, feature-level evidence, protected-attribute exclusion and human review instead of presenting a black-box score.

15. References (APA 7th Edition)

1. Wang, J., Xu, Y., Liu, R., & Du, Y. (2026). Multi-task adversarial learning detects intersectional algorithmic bias in AI recruitment systems. *Scientific Reports*, *16*, 22914. https://doi.org/10.1038/s41598-026-53457-9
2. Kumar, D., Verma, C., & Illés, Z. (2026). Optimizing student job placements with NLP and Explainable AI: A fair and transparent hiring framework. *Array*, *29*, 100729. https://doi.org/10.1016/j.array.2026.100729
3. Spoladore, D., Criscuolo, S., & Isgrò, F. (2026). Representing jobs and job seekers in AI-based recommender systems: A literature review. *Engineering Applications of Artificial Intelligence*, *174*, 114450. https://doi.org/10.1016/j.engappai.2026.114450
4. El-Deeb, R. H., Abdelmoez, W. M., & El-Bendary, N. (2026). Explainable and scalable job recommendation system using transformer-based text summarization, cross-encoder semantic similarity, and ensemble learning. *Procedia Computer Science*, *277*, 1019–1030. https://doi.org/10.1016/j.procs.2026.02.142
5. Agrawal, P., Gandhi, S., Dattani, V., Rachchh, D., Rathod, M., & Vadher, K. (2026). Using machine learning to automate IT job role prediction by resume screening. In J. Choudrie, P. N. Mahalle, T. Perumal, & A. Joshi (Eds.), *ICT for intelligent systems* (Vol. 1517, pp. 71–84). Springer. https://doi.org/10.1007/978-981-96-8895-1_7
6. Zhu, C., Hu, X., Wu, H., Qin, C., Zhu, H., & Xiong, H. (2025). Enhancing job recommendations with LLM-based resume completion: A behavior-denoised alignment approach. *Information Processing & Management*, *62*(6), 104261. https://doi.org/10.1016/j.ipm.2025.104261
7. Shiny, J. J., Sujitha, B., & Mithra, N. (2025). ResML: An improved resume screening and job recommendation system leveraging machine learning. In *2025 2nd Asia Pacific Conference on Innovation in Technology (APCIT)* (pp. 1–6). IEEE. https://doi.org/10.1109/APCIT65661.2025.11411662
8. Singla, P., Kaur, J., Anju, Soni, A., Tuteja, A., & Sharma, S. (2024). Streamlining talent acquisition: A machine learning approach to automated resume screening. In *2024 2nd International Conference on Advanced Computing & Communication Technologies (ICACCTech)* (pp. 1–6). IEEE. https://doi.org/10.1109/ICACCTech65084.2024.00022
9. Surya, C. H. S., Kiriti, G. L. V. S., Saketh, K. S., Charan, P. S., & Anjali, A. (2024). Optimizing job matching through automated resume screening with NLP and machine learning. In *2024 IEEE International Conference on Intelligent Signal Processing and Effective Communication Technologies (INSPECT)* (pp. 877–882). IEEE. https://doi.org/10.1109/INSPECT63485.2024.10896014
10. Python Software Foundation. (n.d.). *Python 3 documentation*. https://docs.python.org/3/

11. Scikit-learn developers. (n.d.). *Scikit-learn user guide*. https://scikit-learn.org/stable/user_guide.html
12. Flask. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/

Project Access Links

- **GitHub Repository:** https://github.com/SKSan2007000/Artificial-Intelligence-CSA1709-Course-Assignment.git
- **Live Vercel Deployment:** https://artificial-intelligence-csa-1709-co.vercel.app/